Spotted: step-by-step build guide for Claude Code

This guide turns your current Expo app into the Spotted design from the prototype. You paste **one step at a time** into Claude Code, check the result on your phone, then move on.

Before you start (5 minutes)

1. **Put the two files into your project.** Copy both into the `docs` folder of your Spotted project:
   - `docs/SPOTTED_BUILD_GUIDE.md` (this file)
   - `docs/spotted-prototype-reference.html` (the prototype's source code; Claude Code reads it for exact colours, data, sizes and animation timings)
2. **Save your current version** so you can always go back. In Claude Code, type:

   > `Commit everything as it is now with the message "Before Spotted redesign".`

3. **Open the app on your phone** with Expo Go (or your development build), so you can watch each change.

How to work through the steps

- Copy the text inside each **Prompt** box into Claude Code, exactly as written.
- When it's done, check the **What you should see** list on your phone.
- If something looks wrong, tell Claude Code what's wrong in plain words, e.g. *"the banner drops too fast, slow it down"*, or attach a screenshot.
- When a step looks right, say **"commit this step"** before starting the next one.
- Don't skip ahead. Each step builds on the one before.

Step 0: Let Claude Code study the design (no code yet)

PROMPT · copy into Claude Code

```
Read docs/SPOTTED_BUILD_GUIDE.md fully, then read docs/spotted-prototype-
reference.html. The HTML file is a design prototype of the Spotted app: the CSS at the
top has all colours, sizes and animations, and the script at the bottom has the sample
data (PLACES, INFL, NEW_OFFERS, LOYALTY) and all the screen logic. Also read
CLAUDE.md, SPEC.md and the README, and look through app/, src/, components/ and
constants/ to understand how this project is set up today.

Do not write any code yet. Reply with:
1. What the project currently has (navigation, screens, libraries).
2. Which parts of the prototype already exist and which are missing.
3. A list of the npm packages you will need to add.
```

```
4. Anything in SPEC.md or CLAUDE.md that conflicts with the prototype.
Keep the existing project structure and conventions. Follow CLAUDE.md.
```

What you should see: a written plan. Read it. If it says it wants to delete or restructure lots of your existing code, ask it why before saying yes.

Step 1: Colours, fonts and sample data

PROMPT · copy into Claude Code

```
Set up the Spotted design foundation, matching docs/spotted-prototype-reference.html:

1. Theme: create a theme file (in constants/ if that's where theme things live) with a
light and a dark palette using the exact values from the .ph and .ph.dark CSS
variables in the prototype (bg, surface, surface2, text, muted, line, brand, btn, acc,
red, chip, map colours). Add a ThemeProvider / hook so every screen can read the
current palette, and store the user's light/dark choice so it survives app restarts.
Default to the phone's system setting.
2. Font: install Poppins with @expo-google-fonts/poppins (400, 500, 600, 700, 800) and
use it everywhere. Keep the splash visible until fonts are loaded.
3. Sample data: copy PLACES, INFL, NEW_OFFERS, LOYALTY, LEVEL and the helper functions
(recsFor, itemsFor, fmt, hasOffers) from the prototype's script into a TypeScript
file, e.g. src/data/mock.ts, with proper types (Place, Influencer, Offer,
LoyaltyCard). All screens will read from here for now; later we swap it for a real
backend.
4. Shared pieces: a small icon set using react-native-svg with the same stroke icons
as the prototype (pin, menu/document, search, sliders, bookmark, map, home, sparkle,
back, close, chevron, play/reel, camera, music note, video, tag, star, chat, bell,
check, moon, sun). Also a PlaceLogo component (gradient circle or square with the 2-
letter monogram, using expo-linear-gradient with c2 → c1).

Rules for the whole project: never use eyes, faces or any human features in icons,
illustrations or animations. Do not use emoji in the UI.
```

What you should see: nothing changes on screen yet, or only the font. That's fine; this is the base.

Step 2: Splash screen (banner unrolls from the top)

PROMPT · copy into Claude Code

```
Build the animated splash screen exactly like the prototype's SPLASH section (see the
.splash, .rod, .bn-*, .road-*, .word, .tagline CSS and keyframes). Use react-native-
reanimated and react-native-svg.

- Full-screen purple radial background (#A14CE8 → #7B2CBF → #5A189A → #3C096C), same
in light and dark mode.
- 0.25s: the dark purple winding road draws itself from left to right (animate
strokeDashoffset), then the white dashed centre line fades in.
```

```
- A dark rod sits on the very top edge of the screen. At 0.9s the red bookmark banner
(with the V notch at the bottom) unrolls DOWNWARD from that rod like a cloth banner on
a building: reveal it from top to bottom (animate its height/clip with an inner mask;
do not squash it), with a red rolled-cloth cylinder travelling down at the reveal edge
and fading out at the bottom, plus horizontal fold shading that fades away.
- 2.05s: the banner sways gently from its top edge (small rotate + skew, settling back
to straight).
- 2.15s: the white location pin pops onto the banner with a bounce.
- 2.6s: "Spotted" rises up in white Poppins ExtraBold (~62px); 2.85s the tagline
"Saved it on a reel. Find it on the road." fades up.
- At ~4.4s go to sign-in (or straight to Home if already signed in). Tapping the
screen skips the intro.
Make the sizes relative to the screen so it looks right on small and large phones.
Respect the phone's Reduce Motion setting (show the final frame instantly).
Also set the native splash (app.json) to the same purple so there's no white flash.
```

What you should see: purple screen, road draws in, red banner unrolls down from the top edge and sways, pin pops, "Spotted" appears, then it moves on by itself.

Step 3: Sign in with phone number

PROMPT · copy into Claude Code

```
Build the sign-in flow from the prototype's SIGN IN section, as three screens (or
three steps of one screen):

1. Phone: purple header with the small banner and road, then a white sheet sliding up
with "Welcome to Spotted", a phone field with a UAE flag + "+971" country picker,
"Send code" button, and the terms line.
2. Code: "Enter the code", 6 boxes that fill one by one (one hidden TextInput with
keyboardType number-pad and textContentType/autoComplete one-time-code so iOS/Android
can auto-fill the SMS code). Auto-check when 6 digits are entered. Wrong code: the
boxes shake and show "That code didn't match. Please try again." Include a "Resend
code" countdown and "Change number".
3. Details (first-time users only): Name and Email, then "Start exploring" → Home.

For now, fake the SMS: accept 123456 as the correct code and keep it behind an auth
service file (e.g. src/services/auth.ts with sendCode(phone) and verifyCode(phone,
code)) so we can plug in a real provider later. Save the signed-in user (name, email,
phone) on the device so the user stays signed in.
```

What you should see: phone number, then code (123456 works, anything else shakes), then name and email, then the app.

Step 4: Bottom tab bar

PROMPT · copy into Claude Code

```
Replace the current tab layout with the Spotted bottom bar (prototype: .nav, .nb, .nb-home, .nav-ind, .nb-dot) using expo-router Tabs with a custom tabBar component.

Order: Saved · Map · Home (centre) · New In · Profile. Home opens first.
- Home is a raised purple gradient circle (#9D4EDD → #5A189A) sitting above the bar, with a border in the background colour.
- The others are an icon + small label. Profile shows a small gradient circle with the user's first initial (no person icon).
- The active icon bounces when tapped and a small dot slides under it to the active tab.
- New In shows a red dot until the user opens that tab.
- Translucent/blurred background (expo-blur), respects the safe area at the bottom.
Create placeholder screens for each tab if they don't exist yet.
```

What you should see: five tabs, Home in the middle as a raised purple button, a dot sliding to the active tab.

Step 5: Home screen

PROMPT · copy into Claude Code

```
Build the Home tab from the prototype's HOME section, top to bottom:

1. Location row: red pin in a rounded square + "You're in" + the user's current area name (use expo-location: ask permission, get position, reverse-geocode to "District, City"; show "Khalifa City, Abu Dhabi" from mock data if permission is denied).
2. "Saved places near you" + "Within 15 km" chip, then a single horizontal row (like Instagram stories) of the user's SAVED places within 15 km, nearest first, maximum 10. Each is a circle with the place logo and, underneath, the distance instead of the name ("1.2 km away"). Ring around each circle slowly rotates: places under 5 km get a bright pink/purple/orange gradient ring with a pulsing glow and bold label; 5–15 km get a softer, muted purple ring. Circles pop in one after another. Tapping opens the Place page.
3. Search bar: placeholder cycles by typing out examples letter by letter ("Try 'Indian'", "Try 'clothes'", "Try 'movies'", ...), grows slightly when focused, purple filter button on the right with a red count badge when filters are on. Active filters show as removable chips under it (the filter sheet itself comes in Step 8; just add the button and chips now).
4. When the user types: search ALL nearby places (not just saved) by name, category, tags and menu items, plus influencers by handle. Show influencer cards first, then place results. Spotted Partner places always come first (with the "Spotted Partner" badge and a purple glow border), then everything else nearest first.
5. When nothing is typed: "Explore nearby" feed of all places within 15 km, Spotted Partners first, then nearest first.

Place card (reuse everywhere): photo/logo thumbnail with a striped awning, name, red bookmark icon if saved, "Category · AED lo–hi pp", distance, badges ("Spotted Partner", "Spotted by N" if influencers recommended it), and a separate "Menu" button on the right. Cards slide up and fade in as they appear.
Use FlatList/FlashList for the long lists so scrolling stays smooth.
```

What you should see: your location, the circle row (bright rings for close places), the search bar typing examples, and the feed. Typing "Indian" or "capital" shows results.

Step 6: Place page and menu PDF

PROMPT · copy into Claude Code

```
Build the Place page (prototype: PLACE PAGE section + .hero, .act, .dd, .rec, .offer
CSS) as a stack screen that slides in from the right.

- Hero image at the top (front entrance and signboard). Use the place's photo URL if
present; otherwise draw the illustrated shop front from the prototype's SVG (awning
stripes and signboard in the place's colours, no people). It slowly zooms out when the
page opens. Back button top-left, "Spotted Partner" badge top-right if partner.
- Name, then "Category · AED lo–hi per person · distance".
- Action row: red pin "Location" button (bounces twice when the page opens) → opens
Google Maps at that place with Linking.openURL("https://www.google.com/maps/search/?
api=1&query=<lat>,<lng>&query_place_id=<googlePlaceId>"); "Menu" button to its right;
save/unsave bookmark button on the far right (fills red when saved, shows a short
toast).
- Dropdown 1 "Influencer recommendations (N)", open by default, animated open/close
with a rotating chevron. Each influencer: round profile circle (their profile picture
URL when we have it, otherwise gradient + initial), handle (tapping opens the
Influencer page), platform + followers, their recommended items each with "AED price",
and UNDER the profile circle a small platform-coloured icon that opens that exact reel
with Linking.openURL(reelUrl).
- Dropdown 2 "Offers (N)", closed by default: "Partner discounts" (Fazaa / Esaad rows
with coloured source badges) and "Spotted offers" (loyalty stamp card and other
Spotted deals). Show friendly empty messages when there are none.

Menu: opens as a bottom sheet / modal that slides up, shows the menu PDF inside the
app with a close (X) button. Add a menuPdfUrl field to Place. Use react-native-webview
to display it (on Android, wrap the URL in a viewer since Android WebView can't show
PDFs directly, or use react-native-pdf if we're on a development build; tell me which
you chose). For mock data with no PDF, render the menu items list styled like a paper
menu, as in the prototype.
```

What you should see: tapping any circle or card opens this page; the dropdowns open and close smoothly; Menu slides up and closes.

Step 7: Influencer page

PROMPT · copy into Claude Code

```
Build the Influencer page (prototype: INFLUENCER PAGE section, .prof-ring, .ptile,
.top7 CSS).

- Top: profile picture in a circle on the LEFT with a rotating gradient ring (zooms in
on open); next to it the handle, followers and niche.
- Under it: a button for each platform they are on (Instagram / TikTok / YouTube),
```

```
with icon + label; tapping opens their profile on that platform via Linking (fall back to the website if the app isn't installed). Buttons pop in one by one.
- "This month's top 7" with a month chip: horizontal swipeable cards numbered 1–7, each with the place image, name, category, distance, and the items they recommend with prices. Cards fan in from the right. Tapping a card opens that Place page.
- "All recommendations": a list of every place they've recommended or made reels about, each with a play button that opens that reel.
Influencers can be found from Home search (Step 5) and from the Place page.
```

What you should see: search "capital", tap the influencer, and you get their page with the top 7 carousel and full list.

Step 8: Filters

PROMPT · copy into Claude Code

```
Build the Filters bottom sheet (prototype: FILTER SHEET section) with @gorhom/bottom-sheet, opened from the filter button on Home search AND on the Saved tab. It uses one shared filter state.

- "How much do you want to spend today?" budget slider PER PERSON, AED 20 to AED 500+ (500 = "Any budget"); the big number updates live as you slide. Use @react-native-community/slider.
- Distance chips: 1 km / 5 km / 10 km / 15 km.
- Category chips: All, Food, Shopping, Fun, Indian, Mexican, Café.
- Toggles: "Spotted by influencers only", "Has offers (Fazaa, Esaad, Spotted)".
- "Reset" at the top, and a big "Show N places" button that shows the live count.
- A place passes the budget filter if its lowest price per person is within the budget.
Apply the filters to Home search results, the Explore feed, and the Saved tab. Show active filters as removable chips under the search bars. Results reshuffle with a smooth layout animation.
```

What you should see: the filter button opens a sheet, the budget number moves with the slider, results update, and chips appear under the search bar.

Step 9: Saved tab

PROMPT · copy into Claude Code

```
Build the Saved tab (prototype: SAVED section, .tile, .minis CSS).

- Title "Saved" + "N places you've spotted".
- Search bar "Search your saved places…" that searches ONLY the user's saved places (e.g. typing "shopping" shows only saved shops), with the same filter button and chips as Home.
- When nothing is typed and no filter is on: folder tiles in a 2-column grid: "All saved", "Food", "Shopping", "Fun". Each tile is a coloured gradient card with up to 3 overlapping place logos and the count. Tiles pop in.
```

```
- Tapping a folder shows that folder's places as place cards with a back arrow +
folder name.
- Saving a place puts it in the folder matching its group (Food / Shopping / Fun).
Saved places must be stored on the device for now (so they survive restarts) through a
small service file so we can move them to the backend later.
```

Step 10: Map tab

PROMPT · copy into Claude Code

```
Build the Map tab with react-native-maps (prototype: MAP section).

- Centre on the user's location with a pulsing blue dot.
- Draw a 15 km circle around the user (dashed purple border, light purple fill).
- Pins: saved places = small red bookmark-shaped markers; Spotted Partners that aren't
saved = purple pins. Pins pop in one by one.
- A floating card at the top: "Map" + "15 km around you" chip + legend (Saved, Spotted
Partner, You).
- Tapping a pin enlarges it and shows a small card above the tab bar with the place
logo, name, category, price, distance and an "Open" button → Place page.
- Use a custom dark map style when the app is in dark mode.
Tell me if Google Maps needs an API key in app.json for Android and how to add it.
```

Step 11: New In tab

PROMPT · copy into Claude Code

```
Build the New In tab (prototype: NEW IN section). Everything is within 15 km of the
user.

1. "New offers": horizontal cards with the place colour/image on top, a shimmering red
"NEW" badge, the source (Fazaa / Esaad / Spotted), the offer text, place name +
distance, and time left in red ("Ends in 3 days"). Tapping opens the Place page with
the Offers dropdown already open.
2. "New influencers on Spotted": circles with profile picture/initial, handle and a
"JUST JOINED" badge → Influencer page.
3. "New places on Spotted": place cards of restaurants/shops that just joined as
Spotted Partners, with "JUST JOINED" + "Spotted Partner" badges.
Sections slide in one after another. Opening this tab clears the red dot on the New In
tab icon.
```

Step 12: Profile tab

PROMPT · copy into Claude Code

```
Build the Profile tab (prototype: PROFILE section, .save-card, .lc, .stamps, .tg CSS).

- Top: circle with the user's initial on a purple-to-red gradient (no photo, no person
icon), name, email, phone number with a green verified tick.
- "You've saved with Spotted" card: purple gradient, big "AED 340" that counts up from
```

```
0 when the tab opens, with a little gold coin burst, and a breakdown: Spotted / Esaad
/ Fazaa.
- "My loyalty cards": swipeable cards, one per restaurant, with the logo, "4 of 6
stamps", 6 stamp slots (filled ones press in one by one with a stamp bounce; the next
one glows gold), "Now 10% off" and the next level. Discount rises with stamps up to a
maximum of 25% (use the LEVEL table from the prototype). Tapping a card opens the
place.
- Notifications:
  - "Get new offers on WhatsApp" toggle (green when on). This ONLY controls new offers
being sent to WhatsApp. Save the choice to the user's profile.
  - "Nearby saved-place alerts": these are normal app notifications (not WhatsApp).
  - "Dark mode" toggle wired to the theme from Step 1.
```

Step 13: Nearby alerts ("A place you saved is 1 km away")

This is the key Spotted feature. It needs a **development build**: background location does **not** work in Expo Go.

PROMPT · copy into Claude Code

```
Implement nearby alerts for saved places as LOCAL app notifications (not WhatsApp):

- Use expo-location geofencing (startGeofencingAsync) with expo-task-manager, and
expo-notifications for the alert.
- Register geofences for the user's saved places nearest to them (iOS allows max 20
regions per app, so keep the nearest 20 and refresh the list when the user moves
significantly or saves/unsaves a place).
- On entering a region, send: title "A place you saved is 1.2 km away", body "<Place
name> · <N> influencers recommend it". Tapping the notification opens that Place page.
- Don't alert for the same place more than once a day.
- Permissions: ask for "While using" first with a clear explanation screen, then ask
for "Always" only when the user turns on nearby alerts. Add the iOS and Android
permission texts to app.json (NSLocationAlwaysAndWhenInUseUsageDescription, background
location, notification permissions).
- Also add the in-app banner from the prototype (.notif): it slides down from the top
when an alert arrives while the app is open.
Then tell me the exact commands to make a development build with EAS so I can test
this on my phone.
```

What you should see: after installing the development build, walking or driving near a saved place triggers the notification.

Step 14: Final polish

PROMPT · copy into Claude Code

```
Go through every screen and compare it with docs/spotted-prototype-reference.html in
both light and dark mode. Fix any differences in colours, spacing, font weights,
corner radius and animation timing. Then check:
- Every button is at least 44×44 and has an accessibility label.
```

```
- Text contrast passes in both themes.
- Animations respect Reduce Motion.
- No eyes, faces or human features anywhere; no emoji in the UI.
- Lists scroll smoothly with 100+ places.
Run the project's tests and type-check, and fix anything that fails. List what you
changed.
```

Later: connecting real data (not needed for the design)

The steps above use sample data. When you're ready to go live, these are separate projects to ask Claude Code about one at a time:

Feature	What's needed
Real SMS codes	A phone-auth provider (e.g. Firebase Authentication or Supabase Auth with an SMS service). Replace the fake `verifyCode` in `src/services/auth.ts`.
Places, menus, offers	A backend database (e.g. Supabase or Firebase) with tables for places, menus (PDF links), offers and partner status.
Saving from Instagram	The "Share to Spotted" action from Instagram (a share extension), which reads the reel link and matches it to a place.
Influencer reels and profiles	Store each reel URL and profile link in the database; the app already opens them with links.
WhatsApp offers	The WhatsApp Business Platform (Cloud API) sends offer messages from your server, only to users who switched the toggle on. Message templates need Meta approval.
Fazaa / Esaad offers	Entered by your team or the partner restaurants; Spotted does not need to connect to those apps.

If something goes wrong

- **"It broke something that worked before"**: type *"Undo your last change and try again more carefully"*, or *"go back to the last commit"*.
- **Animation feels off**: describe it: *"banner should take 1.5 seconds and sway less"*.
- **Red error screen on the phone**: copy the error text (or screenshot it) into Claude Code and say *"fix this error"*.
- **Claude Code wants to install something unexpected**: ask *"why do we need this package?"* before agreeing.
- **Expo Go says a module isn't supported**: that feature needs a development build; ask Claude Code *"set up an EAS development build"*.